

Nghiên cứu Động lực Nội tại trong Học Tăng cường

Correlation-Aware Reward Exploration (CARE)

Đoàn Văn Giáp

Giảng viên hướng dẫn: TS. Nguyễn Thị Thủy

Trường Đại học Công nghệ - Đại học Quốc gia Hà Nội

Tháng 5, 2026

Khóa luận tốt nghiệp – Khoa Công nghệ Thông tin

Học tăng cường (RL) đã thành công với phần thưởng dày:

- Atari, AlphaGo, robot điều khiển
- Tác tử nhận tín hiệu thường xuyên $\Rightarrow$ dễ học

Thực tế phần lớn bài toán là sparse reward:

- Tín hiệu chỉ xuất hiện sau chuỗi hành động dài
- Khám phá ngẫu nhiên gần như không bao giờ chạm đích
- Ví dụ: điều hướng, thao tác nhiều bước, mở khóa - lấy đồ

Giải pháp phổ biến

Bổ sung **phần thưởng nội tại** R^I (tò mò, độ mới) vào phần thưởng ngoại sinh R^E :

$$\bar{r}_t = R_t^E + \beta R_t^{I,m}$$

với β là **hệ số cố định**.

Các module nội tại tiêu biểu:
Count-based, ICM, RND, RIDE, RE3.

Vấn đề

Một β duy nhất áp dụng đồng đều cho *mọi trạng thái* của môi trường.

Nhu cầu khám phá phụ thuộc trạng thái:

- Đầu episode, trước cửa chưa mở $\Rightarrow$ cần khám phá mạnh.
- Cuối episode, đã thấy đích $\Rightarrow$ cần khai thác, đập tò mò.
- β cố định không phân biệt được hai loại trạng thái này.

Ba bằng chứng kinh nghiệm:

- β tối ưu **khác nhau** giữa các môi trường (chênh tới $10\times$).
- β tối ưu **thay đổi theo giai đoạn** huấn luyện.
- β **nhạy**: lệch một bậc giá trị có thể làm chính sách suy sụp ($\beta = 0.05$ phá nát hầu hết môi trường trong thí nghiệm).

$\Rightarrow$ Cần một cơ chế **tự điều chỉnh β theo từng trạng thái**, học trực tuyến từ chính kinh nghiệm của tác tử.

Câu hỏi nghiên cứu

Có thể học một hàm $\beta(s_t)$ **phụ thuộc trạng thái**, từ chính trải nghiệm tác tử, để cải thiện cách các module phần thưởng nội tại (count-based, ICM, RIDE) đóng góp vào việc học trong môi trường phần thưởng thưa hay không?

Đóng góp chính:

- 1 Đề xuất **CARE** – Correlation-Aware Reward Exploration: một lớp *scaling* thích ứng, học $\beta_\psi(s_t)$ qua **mục tiêu tương quan** với GAE advantage ngoại sinh.
- 2 **Tính tổng quát theo module**: cùng cơ chế CARE áp dụng cho ba họ nội tại khác nhau (count-based, ICM, RIDE) – không cần thiết kế lại.
- 3 **Đánh giá thực nghiệm 570 lần chạy** trên 6 môi trường MiniGrid: CARE khớp hoặc vượt β cố định tốt nhất ở **7/18 cặp (env, module)** mà không cần grid-search β .

Nền tảng: PPO + 3 module phần thưởng nội tại

PPO (Proximal Policy Optimization):

- Actor–Critic, clipped surrogate
- GAE advantage ($\lambda = 0.95$)
- $K = 80$ epoch / 4,000 bước

Cùng backbone PPO cho mọi cấu hình $\Rightarrow$ so sánh **công bằng** giữa các module nội tại và các chiến lược scaling.

Ba module nội tại được khảo sát

Count	$R^I \propto 1/\sqrt{N(s)}$ qua random projection hash.
ICM	$R^I = \ \hat{\phi}(s') - \phi(s')\ ^2$, lỗi dự báo của <i>forward model</i> trong không gian embedding.
RIDE	$R^I = \ \phi(s') - \phi(s)\ _2 / \sqrt{N_{\text{ep}}(s')}$, "impact-driven" có giảm dần trong episode.

Mọi module đều được *chuẩn hoá z-score* trước khi đưa vào CARE:
 $I^+ = \max(0, (R^I - \mu)/(\sigma + \epsilon)).$

Thay đổi

Thay β cố định bằng một mạng nhỏ $\beta_\psi(s_t)$ – **Beta Network**.

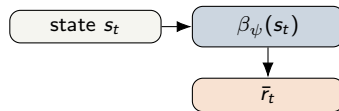
$$\bar{r}_t = R_t^E + \boxed{\beta_\psi(s_t)} \cdot I_t^+, \quad \beta_\psi(s_t) \in [10^{-4}, 5 \times 10^{-2}]$$

Kiến trúc Beta Network:

- Encoder 2 lớp (256) + đầu ra 128
- Sinh ra $\log \beta_\psi$, clip để $\beta \in [\beta_{\min}, \beta_{\max}]$
- Anchor $\beta_0 = \sqrt{\beta_{\min} \beta_{\max}} \approx 2.24 \times 10^{-3}$ (trung bình hình học = trung điểm log-space)
- Adam, lr 5×10^{-4} , 1 bước ψ /PPO update

Tín hiệu giám sát: tương quan với extrinsic advantage.

Trực giác: *khuếch đại tò mò ở những trạng thái mà chính sách đang vượt kỳ vọng giá trị; dập tò mò ở trạng thái còn lại.*



(dùng trực tiếp, không Polyak target)

Meta-loss của Beta Network (correlation + log-space regularizer):

$$\mathcal{L}_\beta(\psi) = -\mathbb{E}[\hat{I}_z \cdot \hat{A}_z^E] + \lambda_{\text{reg}} (\log \beta_\psi(s) - \log \beta_0)^2$$

- $\hat{I} = \beta_\psi(s) \cdot I^+$: phần thưởng nội tại đã có trọng số.
- $\hat{A}^E$: GAE advantage ngoại sinh **thô**, chưa chuẩn hoá.
- $(\cdot)_z$: z-score trong minibatch $\Rightarrow$ tương quan Pearson trong batch.
- $\lambda_{\text{reg}} = 10^{-3}$: kéo β_ψ về anchor β_0 khi không có tín hiệu.

Diễn giải gradient

$\partial \mathcal{L}_\beta / \partial \log \beta_\psi(s)$ tỉ lệ với covariance trong batch giữa $I^+(s)$ và $\hat{A}^E(s)$:

- **Cov dương** (intrinsic cao đi đôi với advantage dương) $\Rightarrow$ *tăng* β .
- **Cov âm** (intrinsic cao đi đôi với advantage âm) $\Rightarrow$ *giảm* β .
- **Cov ≈ 0** (cold-start, sparse) $\Rightarrow$ regularizer chiếm ưu thế, $\beta \rightarrow \beta_0$.

Tại sao dùng advantage thay vì return?

Một lựa chọn thiết kế quan trọng của CARE: tín hiệu giám sát cho Beta Network là **GAE advantage** $\hat{A}^E$, không phải *discounted return* G^E .

Vấn đề của G^E :

- Variance bị chi phối bởi *vị trí trong episode*.
- Trạng thái đầu episode "thấy" gần như toàn bộ $\sum \gamma^k r_k$.
- Trạng thái cuối episode "thấy" hầu như 0.
- $\Rightarrow$ tín hiệu giả: state đầu luôn có return cao hơn, không phản ánh đóng góp riêng của state.

Lợi ích của $\hat{A}^E = G^E - V(s)$:

- Trừ baseline $V(s)$ **loại bỏ confound vị trí episode**.
- Tín hiệu là "vượt kỳ vọng baseline bao nhiêu" – credit per-state sạch hơn.
- Phù hợp đúng với trực giác "khuếch đại nội tại nơi chính sách đang vượt baseline".

Trách nhiệm cài đặt

Beta Network nhận **advantage thô (un-normalized)** để giữ thông tin độ lớn tín hiệu. PPO policy update vẫn dùng advantage đã chuẩn hoá như chuẩn mực.

6 môi trường MiniGrid:

- DoorKey-8x8 – chìa → cửa → đích
- Empty-16x16 – thưởng cực thưa
- KeyCorridor-S3R3 – nhiều phòng, nhiều chìa
- LavaCrossing-S9N3 – né lava
- RedBlueDoors-8x8 – thứ tự cửa
- UnlockPickup – mở khóa rồi nhặt đồ

19 cấu hình $\times$ 5 seeds $\times$ 6 envs = 570 lần chạy:

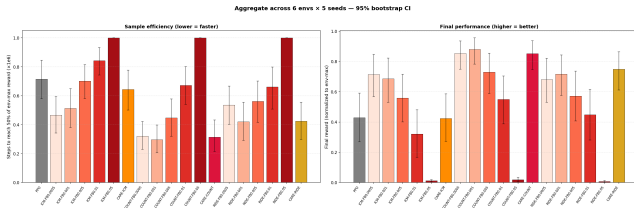
- B_0 : PPO không nội tại (kiểm soát)
- $F_{m,\beta}$: 3 module $\times$ 5 giá trị β cố định
 $\beta \in \{0.0005, 0.001, 0.005, 0.01, 0.05\}$
- C_m : 3 biến thể CARE (count/ICM/RIDE)

Mỗi seed: 10^6 bước môi trường, $\sim 2-4$ giờ trên T4.

Bốn chỉ số đánh giá

Episode return	Mean reward 10 episode cuối, trung bình 5 seeds.
Sample efficiency	Số bước đầu tiên đạt $0.7 \times$ best asymptotic.
Learning stability	Std-dev chéo seed của reward curve.
Learned β_ψ	Phân phối + heatmap theo vị trí tác tử.

Kết quả: CARE-COUNT & CARE-RIDE – khớp hoặc vượt



Hình: Tổng hợp sample-efficiency (nhỏ tốt) và normalized final reward (lớn tốt) trên 6 môi trường.

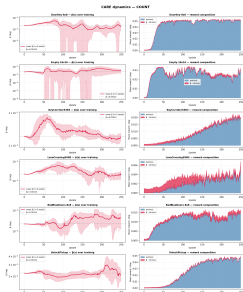
Tổng quan 18 cặp (env, module):

- CARE khớp hoặc vượt β cố định tốt nhất: 7/18 (gồm 3 vượt và 4 hoà ± 0.005).
- Kém hơn: 11/18 (đa số ở ICM và RedBlueDoors).

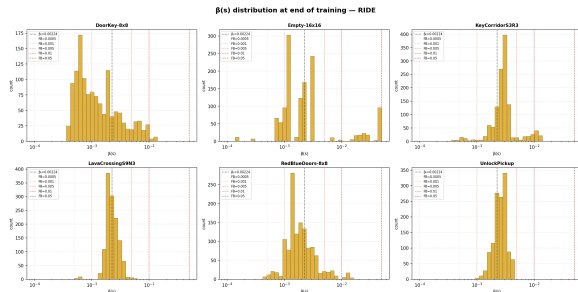
Điểm sáng:

- CARE-RIDE trên UnlockPickup: 0.536 vs 0.369 (+0.167).
- CARE-COUNT trên Empty-16x16: chạm ngưỡng 66k bước (vs 82k tốt nhất của β cố định).
- Variance thấp nhất trên mọi env mà CARE-COUNT hội tụ.

Kết quả: $\beta_\psi(s)$ thật sự phụ thuộc trạng thái



CARE-COUNT: mean $\beta_\psi(s_t)$ qua thời gian.

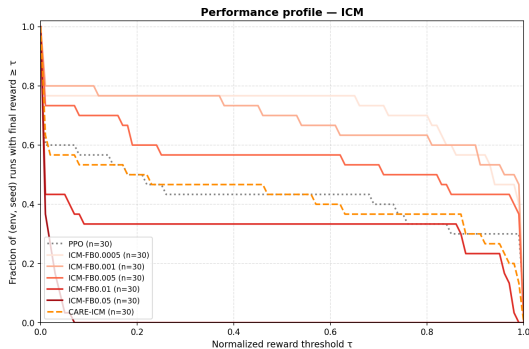


CARE-RIDE: histogram β_ψ cuối training.

Ba chế độ quan sát được:

- 1 **Extrinsic sớm (DoorKey, Empty):** β_ψ tách khỏi β_0 trong ~ 10 update đầu, giảm về $\beta_{\min}$.
- 2 **Extrinsic giữa training (KeyCorridor, RedBlueDoors, UnlockPickup):** β_ψ neo tại β_0 cold-start, chỉ tách khi advantage có magnitude.
- 3 **Phân phối cuối β_ψ trải ~ 1 bậc giá trị** qua các state – bằng chứng học được hàm *thật sự phụ thuộc trạng thái*, không collapse về một hằng.

Phân tích thất bại: vì sao CARE-ICM kém?



Performance profile cho module ICM – CARE bị nhiều β cố định nhỏ vượt.

Trường hợp tiêu biểu: KeyCorridor.

- Best fixed- β (ICM, $\beta=0.001$): 0.457.
- CARE-ICM: **0.000** (cả 5 seed).

Cơ chế thất bại:

- Lỗi forward-model ICM *không nhất thiết* nằm trên đường tới đích.
- Vùng có I^+ lớn nhưng advantage cũng dương ngẫu nhiên $\Rightarrow$ cov dương *nhầm*.
- Gradient $\mathcal{L}_\beta$ **khuếch đại** β_ψ ở vùng *ngoài* đường tới đích.
- Histogram CARE-ICM trên KeyCorridor: **bimodal**, đuôi **nặng gần** $\beta_{\max}$.

Bài học

CARE **không phải** thay thế cho việc chọn module nội tại tốt. Đó là một bộ nhân phụ thuộc trạng thái mà chất lượng phụ thuộc vào *độ tương quan* giữa tín hiệu nội tại thô và extrinsic-success path.

Đóng góp đã được kiểm chứng:

- Một cơ chế **scaling thích ứng theo trạng thái** cho phần thưởng nội tại, được huấn luyện qua mục tiêu **tương quan với GAE advantage**.
- Cùng cơ chế CARE áp dụng được cho **3 họ module**, không cần thiết kế lại.
- Trên **count-based** và **RIDE**: CARE khớp/vượt β cố định tốt nhất *không cần* grid-search.

Giới hạn:

- **Phụ thuộc chất lượng module**: CARE không cứu được ICM khi tín hiệu thô không khớp đường-tối-dịch (KeyCorridor: 0.000).
- **Cold-start trên môi trường cực thưa** (Empty-16x16): advantage gần như 0 ở mọi state, gradient correlation noise, β_ψ rớt về β_0 – CARE *thoái biến về fixed- β* .
- **Phạm vi MiniGrid**: action rời rạc, observation thấp chiều. Chưa kiểm chứng trên continuous control hoặc môi trường giàu observation.

Hướng mở rộng:

- ➊ **Mở rộng phạm vi:** continuous control (MuJoCo, robotic manipulation), multi-task RL, embodied navigation.
- ➋ **Phân tích lý thuyết sâu hơn** cho mục tiêu tương quan – khi nào adaptive scaling chắc chắn có lợi?
- ➌ **Tín hiệu giám sát thay thế:** bootstrapped novelty, trajectory-level success indicator – khắc phục regime cold-start cực thừa.
- ➍ **Lai ghép:** CARE + meta-gradient method, hoặc CARE + diversity-based exploration.

Xin cảm ơn Hội đồng!

Sẵn sàng nhận câu hỏi.

Đoàn Văn Giáp – juuzou152@gmail.com

[Backup] Bảng đối sánh CARE vs best fixed- β (18/18)

Env	Module	CARE	Best fixed	Best β	Gap
DoorKey-8x8	COUNT	0.973	0.973	0.0005	+0.001
	ICM	0.770	0.973	0.0005	-0.202
	RIDE	0.973	0.973	0.0005	+0.000
Empty-16x16	COUNT	0.959	0.975	0.0005	-0.017
	ICM	0.575	0.975	0.0005	-0.400
	RIDE	0.941	0.974	0.001	-0.033
KeyCorridor	COUNT	0.831	0.828	0.001	+0.003
	ICM	0.000	0.457	0.001	-0.457
	RIDE	0.548	0.703	0.001	-0.155
LavaCrossing	COUNT	0.372	0.530	0.001	-0.158
	ICM	0.077	0.060	0.001	+0.017
	RIDE	0.543	0.512	0.0005	+0.031
RedBlueDoors	COUNT	0.653	0.779	0.0005	-0.126
	ICM	0.273	0.737	0.0005	-0.464
	RIDE	0.593	0.794	0.001	-0.201
UnlockPickup	COUNT	0.936	0.936	0.0005	+0.000
	ICM	0.716	0.937	0.001	-0.221
	RIDE	0.536	0.369	0.0005	+0.167

[Backup] Pseudocode CARE (Algorithm 1)

- ➊ Khởi tạo π_θ , V_ϕ , Beta Network β_ψ , intrinsic module m .
- ➋ Lặp đến T bước:
 - ➊ Thu thập rollout $\{(s_t, a_t, r_t^E, s_{t+1})\}$ với π_θ .
 - ➋ Tính $R_t^{I,m}$ từ module m ; chuẩn hoá I_t^+ .
 - ➌ Tính $\beta_\psi(s_t)$ và shaped reward $\bar{r}_t = r_t^E + \beta_\psi(s_t) \cdot I_t^+$.
 - ➍ Tính GAE advantage $\hat{A}_t^E$ thô (từ extrinsic) và $\hat{A}_t^{\text{shaped}}$ (cho policy).
 - ➎ Cập nhật π_θ , V_ϕ bằng PPO ($K=80$ epoch, $\hat{A}$ chuẩn hoá).
 - ➏ Một bước Adam trên ψ : $\mathcal{L}_\beta = -\mathbb{E}[\hat{I}_z \hat{A}_z^E] + \lambda_{\text{reg}}(\log \beta_\psi - \log \beta_0)^2$.
 - ➐ Clip β_ψ vào $[\beta_{\min}, \beta_{\max}]$.

[Backup] Hyperparameter chính

PPO	$K = 80, \epsilon_{\text{clip}} = 0.2, \gamma = 0.99, \lambda_{\text{GAE}} = 0.95$ $\text{lr}_{\text{actor}} = 3 \times 10^{-4}, \text{lr}_{\text{critic}} = 10^{-3}, \text{rollout } T = 4,000$
CARE	$\beta_{\min} = 10^{-4}, \beta_{\max} = 5 \times 10^{-2}$ $\beta_0 = \sqrt{\beta_{\min} \beta_{\max}} \approx 2.24 \times 10^{-3}$ $\lambda_{\text{reg}} = 10^{-3}, \text{lr}_{\psi} = 5 \times 10^{-4}, \text{weight-decay } 10^{-6}$ 1 bước ψ /PPO update, grad-clip L2 1.0, không Polyak target
Sweep β cố định	$\beta \in \{0.0005, 0.001, 0.005, 0.01, 0.05\}$
Seeds	$\{1, 2, 3, 4, 5\}$, mỗi run 10^6 bước